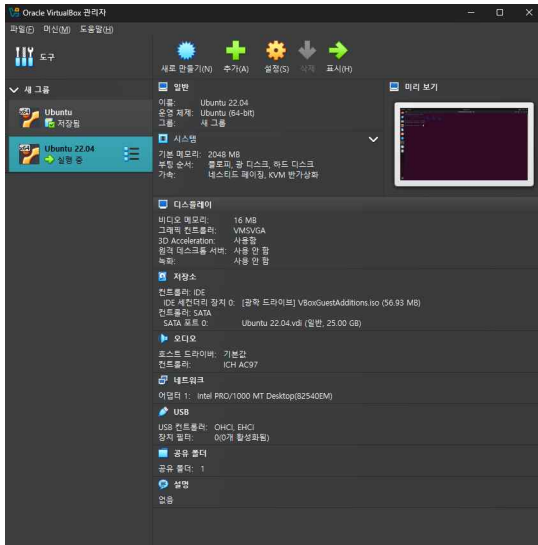


0. Linux를 설치하시오



방법2) 가상 머신으로 Linux를 설치하였습니다.

1. 리눅스 커널 및 리눅스 배포판의 이해

1-(1). 리눅스 배포판, 리눅스 커널, Ubuntu LTS 버전, Ubuntu 정규버전 용어를 설명하시오

리눅스 배포판: 리눅스 커널을 기반으로 필요한 유틸리티, 라이브러리, 패키지 관리자 등을 포함하여 하나의 완전한 운영체제로 만든 패키지를 의미한다. 리눅스 배포판의 예시로는 리눅스 우분투(Ubuntu), 리눅스 데비안(Debian) 등이 있다.

리눅스 커널: 리눅스 운영체제의 핵심 부분(커널)으로, 하드웨어와 소프트웨어 사이에서 동작하는 역할을 수행한다. 사용자가 컴퓨터 HW를 쉽게 돌릴 수 있게 하는 SW라고 생각할 수도 있다. 커널이 하는 역할은 프로세스 관리(CPU 자원 분배), 메모리 관리, 파일 시스템 관리 등이 있다.

Ubuntu LTS 버전: 리눅스 Ubuntu 배포판에서 장기간 안정적 지원을 보장하는 버전을 의미한다. Long Term Support의 줄임말이다. Ubuntu LTS 버전은 안정성과 호환성 위주이기 때문에, 서버나 기업 환경, 연구소 등에서 많이 사용한다는 특징이 있다. 예를 들어 Ubuntu 20.04 LTS, Ubuntu 22.04 LTS 버전 등이 있고, 출시 주기는 2년에 한 번씩 출시한다

Ubuntu 정규 버전: Ubuntu에서 LTS가 아닌 일반 버전(Interim Release)을 뜻한다. 특징은 최신 기능과 최신 패키지를 빠르게 제공하기 때문에, 최신 기술 테스트를 원하는 개발자들에게 적합하다는 특징이 있다. 하지만 안정성이 떨어질 수 있기 때문에 장기 사용에는 적합하지 않다.

2. /proc 파일 시스템의 이해

2-(1). more /proc/cpuinfo 명령을 실행하여 캡처하고 해당 명령어가 무엇을 실행한 것인지 설명하시오

more /proc/cpuinfo:

/proc/cpuinfo는 리눅스에서 CPU 관련 정보를 담고 있는 가상 파일이다. CPU 모델명, 클럭 주파수, 캐시 크기, 지원 기능 등 CPU 사양을 확인할 때 사용된다. 그리고 more 명령어는 터미널에서 긴 텍스트 파일을 페이지 단위로 출력하는 프로그램이다.

따라서 more /proc/cpuinfo는 CPU의 상세 사양과 상태 정보를 페이지 단위로 보여주는 명령어이다

```
seoknam@seoknam-VirtualBox:~$ more /proc/cpuinfo
processor       : 0
vendor_id     : GenuineIntel
cpu family    : 0
model        : 186
model name    : 13th Gen Intel(R) Core(TM) i7-1360P
stepping      : 2
microcode    : 0xffffffff
cpu MHz      : 2011.210
cache size   : 18432 KB
physical id  : 0
siblings     : 1
core id      : 0
cpu cores    : 1
apicid       : 0
initial apicid : 0
fpu          : yes
fpu_exception : yes
cpuid level  : 22
wp           : yes
flags        : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx rdtscp lm constant_tsc rep_good nopl xtopology nonstop_tsc cpuid tsc_known_freq pni pclm
uqldq ssse3 cx16 sse4_1 sse4_2 movbe popcnt aes rdrand hypervisorlahf_lahf_lm abm 3dnowprefetch lbrs_enhanced fsgsbase bti1 bti2 invpcid rdseed adx clflushopt sha_ni arat md_clear flush_lid arch_capabilities
bugs         : spectre_v1 spectre_v2 spec_store_bypass swappgs retbleed e1bs_prrsb rdfs bhi
bogomips     : 5222.42
clflush size : 64
cache alignment : 64
address sizes : 39 bits physical, 48 bits virtual
power management:
```

2-(2). (학생의) 컴퓨터 시스템에 있는 프로세서의 개수는 몇 개인지 쓰시오.

```
seoknam@seoknam-VirtualBox:/proc$ lscpu | grep "^CPU(s):"
CPU(s): 1
```

1개(가상머신 OS가 인식하는 논리 프로세서 개수. VirtualBox에 논리 CPU 1개만 할당된 상태)

2-(3). 각 프로세서의 frequency(CPU 사용 비율)는 얼마인지 쓰시오

```
cpu MHz : 2611.210
```

cpu MHz: 2611.210 MHz로 확인됨.

(2611.210 MHz = 2.61 GHz)

```
seoknam@seoknam-VirtualBox:/proc$ more /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 186
model name     : 13th Gen Intel(R) Core(TM) i7-1360P
stepping       : 2
microcode      : 0xffffffff
cpu MHz        : 2611.210
```

2-(4). (학생의) 컴퓨터 시스템의 물리적 메모리 크기는 얼마인지 쓰시오.

```
seoknam@seoknam-VirtualBox:/proc$ more /proc/meminfo
MemTotal: 2015832 kB
MemFree: 419360 kB
MemAvailable: 1208020 kB
Buffers: 53336 kB
Cached: 834668 kB
SwapCached: 38996 kB
Active: 692980 kB
Inactive: 458396 kB
Active(anon): 166732 kB
Inactive(anon): 106812 kB
Active(file): 526248 kB
Inactive(file): 351584 kB
Unevictable: 0 kB
Mlocked: 0 kB
SwapTotal: 2693116 kB
SwapFree: 2286504 kB
Zswap: 0 kB
Zswapped: 0 kB
Dirty: 0 kB
Writeback: 0 kB
AnonPages: 250412 kB
Mapped: 195168 kB
Shmem: 10172 kB
KReclaimable: 84688 kB
Slab: 196296 kB
SReclaimable: 84688 kB
SUnreclaim: 111608 kB
KernelStack: 7004 kB
PageTables: 16972 kB
SecPageTables: 0 kB
NFS_Unstable: 0 kB
Bounce: 0 kB
WritebackTmp: 0 kB
CommitLimit: 3701032 kB
Committed_AS: 3582980 kB
VmallocTotal: 34359738367 kB
```

해당 컴퓨터 시스템(가상 머신)에 할당된 물리적 메모리 크기: 2015832KB -> 2015MB -> 약 2GB

2-(5). 위 물리적 메모리 중 free한 크기는 얼마인지 쓰시오.

MemFree 419360KB -> 약 420MB 정도가 free한 크기이다

(free = 현재 사용되지 않고 남아있는 메모리)

2-(6). 시스템 부팅 후 fork()된 프로세스의 개수는 몇 개인지 쓰시오.

```
seoknam@seoknam-VirtualBox:/proc$ cat /proc/stat | grep processes
processes 28549
```

/proc/stat 파일의 processes값 = 부팅 이후 지금까지 생성된 모든 프로세스의 개수

즉 시스템 부팅 후 28549번 fork()되어 프로세스가 생성되었다

2-(7). 부팅 후 문맥 교환(context switch)가 이루어진 횟수는 얼마인지 쓰시오.

```
seoknam@seoknam-VirtualBox:/proc$ cat /proc/stat | grep ctxt
ctxt 1637859
```

/proc/stat의 ctxt값 = 부팅 이후 일어난 context switch 횟수

즉 시스템 부팅 후 1637859번 context switch가 이루어졌다

3. 실행 중인 프로세스의 상태 모니터링

3-(1). 주어진 cpu.c를 컴파일하고 아래 명령어를 실행하고 top 명령어 실행하고 그 결과를 캡처하시오.

```
top - 13:12:57 up 3:33, 1 user, load average: 0.37, 0.09, 0.07
작업: 188 총계, 2 실행, 186 대기, 0 잠중, 0 준비
NCpus: 96.8 us, 3.2 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 st, 0.0 st
MEM 메모리: 1968.6 total, 145.0 free, 665.6 used, 1158.0 buff/cache
MEM 스왑: 2630.0 total, 2236.9 free, 393.1 used, 1114.3 avail 메모리

  PID USER      PR  NI    VIRT    RES    SHR  S  %CPU  %MEM     TIME+ COMMAND
28669 seoknam  20   0   2644    896    896  R   90.3   0.0   0:23.69 cpu
1274 seoknam  20   0 4039732 213636 118456  S   0.3  10.6   0:49.20 gnome-shell
28681 seoknam  20   0   12040   4352   3584  R   0.3   0.2   0:00.03 top
  1 root      20   0 168816   13216   8352  S   0.0   0.7   0:03.72 systemd
  2 root      20   0      0      0      0  S   0.0   0.0   0:00.00 kthread
  3 root      20   0      0      0      0  S   0.0   0.0   0:00.00 pool_workqueue_release
  4 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-rcu_g
  5 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-rcu_p
  6 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-slub
  7 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-netns
 12 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-np_pe
 13 root      20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_kthread
 14 root      20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
 15 root      20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
 16 root      20   0      0      0      0  S   0.0   0.0   0:02.09 ksoftirqd/0
 17 root      20   0      0      0      0  I   0.0   0.0   0:01.46 rcu_preempt
 18 root      rt   0      0      0      0  S   0.0   0.0   0:00.23 migration/0
 19 root     -51   0      0      0      0  S   0.0   0.0   0:00.00 idle_inject/0
 20 root      20   0      0      0      0  S   0.0   0.0   0:00.00 cpuhp/0
 21 root      20   0      0      0      0  S   0.0   0.0   0:00.00 kdevtmpfs
 22 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-net_
 24 root      20   0      0      0      0  S   0.0   0.0   0:00.00 kauditd
 25 root      20   0      0      0      0  S   0.0   0.0   0:00.01 khungtaskd
 26 root      20   0      0      0      0  S   0.0   0.0   0:00.00 oom_reaper
 27 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-write
 28 root      20   0      0      0      0  S   0.0   0.0   0:00.99 kcompactd0
 29 root      25   5      0      0      0  S   0.0   0.0   0:00.00 ksm
 30 root      39  19      0      0      0  S   0.0   0.0   0:00.00 khugepaged
 31 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-kint
 32 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-kbloc
 33 root      0 -20      0      0      0  I   0.0   0.0   0:00.00 kworker/R-blkcg
 35 root     -51   0      0      0      0  S   0.0   0.0   0:00.00 irq/9-acpi
```

3-(2). cpu 명령을 실행하는 프로세스의 PID는 무엇인가 설명하시오.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
28669	seoknam	20	0	2644	896	896	R	97.7	0.0	1:25.60	cpu

cpu 명령을 실행하는 프로세스의 PID = 28669

PID는 Process ID의 줄임말이다.

COMMAND를 보면 cpu이므로, cpu 명령을 실행하는 프로세스의 PID는 28669임을 알 수 있다.

3-(3). 위 프로세스는 얼마나 많은 CPU와 메모리를 소비하는지 쓰시오.

CPU 소비량 -> %CPU: 97.7의 CPU를 소비하고 있다

메모리 소비량 -> top 명령어의 %MEM를 보면 0.0이다. 하지만 이는 백분율로 표현시 소수점 첫째 자리까지만 표현해서 0.0이 나온 것이라 판단했다. 그래서 더 정밀한 확인을 위해 아래 명령어를 사용했다.

```
seoknam@seoknam-VirtualBox:~/OS/OS_1$ ps -p 28669 -o pid,vsz,rss,pmem,comm

  PID    VSZ    RSS %MEM COMMAND
 28669   2644    896   0.0  cpu
```

VSZ(Virtual Memory Size): 해당 프로세스가 사용 중인 가상 메모리 공간의 전체 크기 = 2644KB

RSS(Resident Set Size): 실제로 물리 메모리(RAM)에 상주하는 크기 = 896KB

즉 RSS는 실제 메모리 소비량에 가깝다.

위 결과를 보아 ./cpu 프로세스는 메모리 사용량이 매우 작고, CPU를 많이 쓰는 프로그램임을 알 수 있다.

3-(4). 위 프로세스의 현재 상태는 무엇인지 확인하시오. 예를 들어 실행 중, 차단(block) 중, 잠비 등

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
28669	seoknam	20	0	2644	896	896	R	97.7	0.0	1:25.60	cpu

S: R값을 보아 Running(실행 중) 상태임을 알 수 있다.

(Z: Zombie, T: Stopped)

- 4. 새로운 자식 프로세스를 생성하여 사용자 명령을 실행하는 방법 이해

(1) cpu-print.c를 컴파일하고 셸에서 아래 명령 차례로 실행하고 그 결과를 캡처하시오.

\$ gcc cpu-print.c -o cpu-print

\$./cpu-print

※ 참고 : 무한 루프 수행 중 다른 터미널을 열고 ps 명령을 실행

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ps -aux | grep cpu-print
seoknam    30608 29.5  0.0   2776 1408 pts/0    S+   13:48   0:03 ./cpu-print
```

(2) cpu-print 프로세스의 부모, 즉 셸 프로세스의 PID는 무엇인지 쓰시오. 이 pid로부터 5세대 이상의 조상 프로세스 (또는 init 프로세스)에 도달 할 때까지 거슬러 모든 조상의 PID는 무엇인지 쓰시오.

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ps -ef | grep cpu-print
seoknam    30608    1990 29 13:48 pts/0    00:00:24 ./cpu-print
```

cpu-print 프로세스의 부모, 즉 셸 프로세스의 PID = 1990이다.
아래 사진을 보면 셸 프로세스(bash)의 PID=1990임을 알 수 있다.

```
seoknam    1990  0.0  0.2 12864 4608 pts/0    Ss   09:40   0:00 bash
```

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ps -ef | grep bash
seoknam    1990    1964  0 09:40 pts/0    00:00:00 bash
```

PID=1990 프로세스(bash)의 부모 프로세스 PID는 1964이다.

```
seoknam    1964  0.9  3.0 644620 61668 ?        Rsl  09:40   2:24 /usr/libexec/gnome-terminal-server
```

PID=1964 프로세스의 이름은 /usr/libexec/gnome-terminal-server이다.

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ps -ef | grep /usr/libexec/gnome-terminal-server
seoknam    1964    1118  2 09:40 ?        00:05:06 /usr/libexec/gnome-terminal-server
```

PID=1964 프로세스의 부모 프로세스 PID는 1118이다.

```
seoknam    1118  0.0  0.4 19764 9652 ?        Ss   09:39   0:00 /lib/systemd/systemd --user
```

PID=1118 프로세스의 이름은 /lib/systemd/systemd --user이다

```
seoknam    1118      1  0 09:39 ?        00:00:00 /lib/systemd/systemd --user
```

PID=1118 프로세스의 부모 프로세스 PID는 1이다

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ps -aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.6 168016 12960 ?        Rs   09:39   0:03 /sbin/init splash
```

PID=1인 프로세스의 이름은 /sbin/init splash이다
이 프로세스는 시스템 부팅 시 최초로 실행되는 프로세스(init/systemd)이다.

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ps -ef | grep /sbin/init
root         1      0  0 09:39 ?        00:00:03 /sbin/init splash
```

결론적으로 전체 조상 체인을 적어보면,

cpu-print(PID=30608)

-> 부모: bash(PID=1990)

-> 조부모: /usr/libexec/gnome-terminal-server(PID=1964)

-> 증조부: /lib/systemd/systemd --user(PID=1118)

-> 최상위: /sbin/init(PID=1)

(3) \$./cpu-print > /tmp/tmp.txt & 을 실행하고 새로 생성 된 프로세스의 proc 파일 시스템 정보를 캡처하시오. file descriptor 0, 1 및 2 (표준 입력, 출력 및 에러)가 가리키는 위치에 주의.

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ./cpu-print > /tmp/tmp.txt &
[1] 30791
seoknam@seoknam-VirtualBox:~/05/05_1$ ls -l /proc/30791/fd
합계 0
lrwx----- 1 seoknam seoknam 64  9월  4 14:10 0 -> /dev/pts/0
l-wx----- 1 seoknam seoknam 64  9월  4 14:10 1 -> /tmp/tmp.txt
lrwx----- 1 seoknam seoknam 64  9월  4 14:10 2 -> /dev/pts/0
```

해당 백그라운드 프로세스가 실행되면 /proc 밑에 해당 프로세스의 디렉토리 또한 만들어진다.

ls -l /proc/<PID>/fd를 통해 해당 프로세스의 오픈된 파일 디스크립터들을 확인할 수 있다

FD0(표준 입력) -> 현재 터미널(/dev/pts/0)

FD1(표준 출력) -> 리다이렉트된 /tmp/tmp.txt

FD2(표준 에러) -> 현재 터미널(/dev/pts/0)

(4) 위 이러한 내용을 토대로, 셸에서 I/O 리디렉션을 구현하는 방법을 설명하시오.

우선 /proc/<PID>/fd는 해당 프로세스의 PD 테이블의 상태를 보여주기 때문에, 리디렉션이 어떻게 적용됐는지 확인할 수 있다.

셸에서 I/O 리디렉션을 구현하는 방법을 설명하기 위해서 I/O 리디렉션의 과정을 생각해 보았다.

셸에서는 >, <, 2>, >> 같은 리디렉션 연산자를 만나면, 파일 디스크립터의 연결을 바꿔준다.

예를 들어 \$./cpu-print > /tmp/tmp.txt &를 하면, stdout(1)이 원래 터미널(/dev/pts/0)을 가리키다가 /tmp/tmp.txt로 바뀐다.

이는 셸 내부의 구현 관점으로 보면,

fork() 호출

-> 자식 프로세스에서 리디렉션 처리: int fd = open("/tmp/tmp.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

-> dup2()를 사용해 기존 FD를 새 파일 디스크립터로 교체: dup2(fd, STDOUT_FILENO); close(fd);

-> 실제 프로그램(./cpu-print)을 execvp() 등으로 실행(이때, 표준 입출력 FD는 이미 바뀐 상태이므로 프로그램은 수정 없이도 리디렉션 적용됨)

정도로 생각해 볼 수 있다.

따라서 셸의 I/O 리디렉션을 구현하는 것은 결국 파일 디스크립터 테이블 수정 작업이고,

fork() -> open() -> dup2() -> close() -> exec() 순으로 이루어진다고 생각해 볼 수 있다

(5) \$./cpu-print | grep hello & 을 실행하고 새로 생성된 프로세스는 무엇인지 쓰시오.

위 명령어를 실행하면 ./cpu-print, grep이라는 2개의 프로세스가 새로 생성된다.(--color=auto hello는 인자)

(jobs는 현재 셸에서 실행한 백그라운드 잡을 확인하는 명령어이다)

```
seoknam@seoknam-VirtualBox:~/05/05_1$ jobs -l
[1]+ 30828 실행중                  ./cpu-print
    30829                        | grep --color=auto hello &
```

- 5. 프로세스의 가상 메모리와 실제 메모리 비교

5-(1) 주어진 두 프로그램 memory1.c 및 memory2.c를 컴파일하고 실행하고 일부 내용을 캡처하시오.

※ 참고 : 두 프로그램 모두 메모리에 큰 배열을 할당함. 한 프로그램은 할당한 배열에 접근하나 다른 프로그램은 할당한 배열에 접근하지 않음. 두 프로그램 모두 종료하기 전에 일시 중지하여 메모리 사용량을 검사(ps 명령어) 할 수 있음.

```
seoknam@seoknam-VirtualBox:~/05/05_1$ ./memory1

Program : 'memory_1'
_____

PID : 3527
Size of int : 4

Press Enter Key to exit.
```

```
seoknam@seoknam-VirtualBox:~/OS/OS_1$ ./memory2

Program : 'memory_2'

-----

PID : 3528
Size of int : 4

Press Enter Key to exit.
```

5-(2) 각 프로세스의 가상메모리 크기와 실메모리 크기를 캡처해서 비교하고, 주어진 프로그램 소스 코드와 ps 명령어를 통해 확인한 내용을 설명하시오.

```
seoknam@seoknam-VirtualBox:~/OS/OS_1$ ps -o pid,vsz,rss,comm -p 3527

  PID   VSZ   RSS COMMAND
  3527  6564  5376 memory1

seoknam@seoknam-VirtualBox:~/OS/OS_1$ ps -o pid,vsz,rss,comm -p 3528

  PID   VSZ   RSS COMMAND
  3528  6564  5376 memory2
```

ps 명령어를 통해 각 프로세스의 가상메모리 크기와 실메모리 크기를 관찰했을 때 둘 다 메모리 사용량이 비슷하게 나온다. 왜냐하면 memory1.c, memory2.c 코드를 보면 배열을 스택에 잡히도록 int array[ARRAY_SIZE]; 같은 방식으로 선언하는데, 스택은 프로세스가 실행될 때 이미 정해진 크기로 예약되기 때문이다.(malloc()처럼 힙에 공간을 동적으로 생성하는게 아니다) 따라서 두 프로그램 모두 스택에 동일한 크기(1000000*sizeof(int) = 4MB)의 배열이 생긴다. 이때 우선 가상 메모리(VSZ)는 배열 접근 여부와 무관하게 스택에 메모리를 예약했기 때문에 두 프로세스의 VSZ는 동일하다. 그리고 RSS도 거의 비슷하게 보이는 이유는, 배열이 스택에 있으니 커널이 한꺼번에 물리 메모리에 페이지를 할당했기 때문이라 생각한다. (소스코드를 보면 memory1.c는 배열에 접근하지 않고, memory2.c는 배열에 접근하기 때문에 memory2.c의 RSS가 더 높게 나올 것이라 생각했는데, 실제로 확인해 보니 그렇지 않았다. 소스코드에서 배열을 malloc()으로 할당에서 힙 메모리를 할당한 경우 memory1의 RSS가 memory2의 RSS보다 적게 나왔다) 위의 유추한 내용이 맞는지 확인하기 위해, 배열을 malloc()으로 동적 할당한 코드인 memory3.c, memory4.c를 작성하고 테스트 해보았다. 이때 memory3.c는 배열에 접근하지 않고, memory4.c는 배열에 접근한다

```
seoknam@seoknam-VirtualBox:~/OS/OS_1$ ps -o pid,vsz,rss,comm -p 3638

  PID   VSZ   RSS COMMAND
  3638 393404   1408 memory3

seoknam@seoknam-VirtualBox:~/OS/OS_1$ ps -o pid,vsz,rss,comm -p 3639

  PID   VSZ   RSS COMMAND
  3639 393404 392064 memory4
```

확인해보니 힙 공간에 할당된 배열 프로세스의 경우, 배열에 접근하는 memory4 같은 경우에는 실제 메모리가 크게 잡혔지만, 배열에 접근하지 않는 memory3은 실제 메모리가 상대적으로 작게 잡혔다. 그에 반해 가상 메모리는 동일한 크기로 할당되었다.

- 6. 프로세스가 오픈한 파일 그리고 디스크

6-(1) 주어진 disk.c 및 disk1.c 프로그램을 컴파일, 실행하고 일부 내용을 캡처하시오.

```
seoknam@seoknam-VirtualBox:~/OS/OS_1$ gcc disk.c -o disk
seoknam@seoknam-VirtualBox:~/OS/OS_1$ gcc disk1.c -o disk1
```

```
seoknam@seoknam-VirtualBox:~/OS/OS_1$ ls -l disk-files/ | wc -l
5003
```

make-copies.sh를 사용하여 foo.pdf와 다른 파일 이름으로 동일한 내용의 파일을 5,000개 복사하였다

6-(2) disk.c 및 disk1.c 의 실행 파일 프로그램이 실행되는 동안 디스크 사용률을 측정하고, 주어진 프로그램 소스 코드와 iostat 명령어를 통해 확인한 내용을 비교 설명하시오.

사용한 명령어: iostat -d 1: 1초마다 디스크 통계를 출력(누적이 아닌 실시간 측정)

disk.c를 컴파일한 프로그램 실행 시 디스크 사용률

Device	tps	kB_read/s	kB_wrtn/s	kB_dscd/s	kB_read	kB_wrtn	kB_dscd
loop0	0.00	0.00	0.00	0.00	0	0	0
loop1	0.00	0.00	0.00	0.00	0	0	0
loop10	0.00	0.00	0.00	0.00	0	0	0
loop11	0.00	0.00	0.00	0.00	0	0	0
loop12	0.00	0.00	0.00	0.00	0	0	0
loop13	0.00	0.00	0.00	0.00	0	0	0
loop14	0.00	0.00	0.00	0.00	0	0	0
loop2	0.00	0.00	0.00	0.00	0	0	0
loop3	0.00	0.00	0.00	0.00	0	0	0
loop4	0.00	0.00	0.00	0.00	0	0	0
loop5	0.00	0.00	0.00	0.00	0	0	0
loop6	0.00	0.00	0.00	0.00	0	0	0
loop7	0.00	0.00	0.00	0.00	0	0	0
loop8	0.00	0.00	0.00	0.00	0	0	0
loop9	0.00	0.00	0.00	0.00	0	0	0
sda	1939.00	7748.00	28.00	0.00	7748	28	0
sr0	0.00	0.00	0.00	0.00	0	0	0

disk1.c를 컴파일한 프로그램 실행 시 디스크 사용률

Device	tps	kB_read/s	kB_wrtn/s	kB_dscd/s	kB_read	kB_wrtn	kB_dscd
loop0	0.00	0.00	0.00	0.00	0	0	0
loop1	0.00	0.00	0.00	0.00	0	0	0
loop10	0.00	0.00	0.00	0.00	0	0	0
loop11	0.00	0.00	0.00	0.00	0	0	0
loop12	0.00	0.00	0.00	0.00	0	0	0
loop13	0.00	0.00	0.00	0.00	0	0	0
loop14	0.00	0.00	0.00	0.00	0	0	0
loop2	0.00	0.00	0.00	0.00	0	0	0
loop3	0.00	0.00	0.00	0.00	0	0	0
loop4	0.00	0.00	0.00	0.00	0	0	0
loop5	0.00	0.00	0.00	0.00	0	0	0
loop6	0.00	0.00	0.00	0.00	0	0	0
loop7	0.00	0.00	0.00	0.00	0	0	0
loop8	0.00	0.00	0.00	0.00	0	0	0
loop9	0.00	0.00	0.00	0.00	0	0	0
sda	0.00	0.00	0.00	0.00	0	0	0
sr0	0.00	0.00	0.00	0.00	0	0	0

설명:

우선 disk.c는 disk-files 디렉토리에 있는 모든 파일들을 랜덤하게 읽는 프로그램이다. 각 루프마다 새로운 파일을 열기 때문에 tps(초당 I/O 요청 횟수)가 1939.00처럼 높은 값으로 나오고, kB_read/s(초당 읽는 데이터량)가 파일을 읽는 동안은 꾸준히 높은 값이 나오며, %utils(디스크 사용량)이 올라간다. 이때 리눅스는 파일을 한 번 읽으면 해당 내용을 페이지 캐시에 저장한다. 따라서 disk.c 프로그램이 파일들을 읽으면서 페이지 캐시에 읽은 파일들은 저장해둔다. 이때 disk.c 프로그램이 끝나고 disk1.c 프로그램을 실행하면, 두 번째 사진처럼 tps, kB_read/s 등의 값이 0이 나온다. 이는 disk1.c 코드를 보면 foo0.pdf 파일을 반복해서 읽도록 코드가 작성되어 있는데, 이때 foo0.pdf는 disk.c 프로그램이 이미 페이지 캐시에 저장해두었기 때문에, disk1.c 프로그램이 파일을 읽을 때에는 디스크에서 직접 읽는게 아니라 메모리 캐시에서 읽어온다. 따라서 iostat 기준으로는 “디스크 I/O 없음”이 되기 때문에 tps, kB_read/s 등의 값이 0이 되는 것이다.

반복 실험을 해보기 위해서는 메모리 캐시를 비워야 실험이 가능했다. 커널 캐시를 초기화하는 명령어는 아래와 같다.

```
sudo sh -c "echo 3 > /proc/sys/vm/drop_caches"
```

위 명령어를 실행하고 disk.c 또는 disk1.c 프로그램을 실행해야 캐시가 아닌 실제 디스크에서 다시 읽게 된다.